KalshiBot operating manual

Pi checkout: `/home/KalshiBot`
Not financial advice. A wrong contract can go to $0.

What this bot is

KalshiBot is an hourly BTC and ETH threshold scanner. It looks at Kalshi's "will Bitcoin / Ethereum finish above this dollar line this hour?" books (`KXBTCD`, `KXETHD`).

It is not a sports bot, not a 15-minute scalp bot, and not a dashboard. It does not stay running. You start it (or the timer starts it); it scans; if one idea clears the rules it may rest a limit; then it exits.

A contract pays $1 if you are right and $0 if you are wrong. Paying 0.14 means you risk 14¢ per contract to win 86¢. That 14¢ is also the market's implied chance.

The bot's job: estimate a fair chance from spot + recent vol, compare that to the real ask, and only act when leftover edge after fees is large enough. Sitting (`NO_ACTIONABLE_EDGE`) is a valid result.

Time zone

Everything you read is **America/New_York** (EDT in summer, EST in winter): report headers, next settlements, anti-revenge hour, Pi timer (`*:03` Eastern), journal logs, and GitHub issue titles.

Kalshi and Coinbase API calls still send UTC. You should not have to convert.

Prod vs demo

Prod is live Kalshi — real cash (`external-api.kalshi.com`). Demo is paper (`demo-api.kalshi.co`). Your API key was created on kalshi.com, so it only works on prod. Demo returns 401.

`--prod` and `USE_DEMO=false` mean the live host. The prompt `on PROD` means real money.

What one run does

1. Pull BTC and ETH spot from CF Benchmarks BRTI/ERTI via Kalshi (the settlement index). Coinbase is the fallback; Binance often 451s in the US.
2. Estimate hourly vol from recent 1-minute candles (fallback: BTC 0.4%/hour, ETH 0.5%/hour). If vol is 2× typical, sit — that is a news tape, not a range day.
3. Load open hourly above/below books (up to 12 strikes per coin).
4. Score Yes (finishes above) and No (at or below).
5. Fair chance from how far the line is from spot, in typical remaining-hour moves (z-score). It does not assume BTC keeps going up. Spot is a proxy. Kalshi settles on CF Benchmarks RTI (a 60-second average), not Coinbase last tick.
6. Edge is always vs the executable ask, never the mid. The filter uses the taker fee even if it later posts a maker limit.
7. Keep one idea if filters pass. Everything else is watch or avoid.
8. `scan` / `once` only print. `live` posts a post-only GTC limit. Typical ticket $1.50–$2.00, never over $2. If that price would take the book, it steps one tick more passive. It will not market-buy.

Max 1 open hourly ticket (2 only if different coin and opposite side).

Rules it will not break

- Bankroll used for sizing: $40 unless you set `BANKROLL`.
- Preferred risk $1.75, hard cap $2, 5% of bankroll, quarter-Kelly.
- One open hourly idea (do not stack four $2 tickets in one morning).
- Ask between 5¢ and 95¢.
- Need 6% net edge after fees. No 4% exception. Sitting is valid.
- Ban close strikes. Do not fade a line inside ~0.5–0.75% of spot. A fat model number on a tight strike is still a coin-flip.
- |z| > 2.5 needs 8% edge. |z| > 3.5 skip (news-only jump).
- Sit out CPI 8:15–8:45 AM ET and FOMC 1:45–2:45 PM ET on those days. Sit a coin when vol is 2× typical.
- If the last live hourly ticket settled against you, the next idea cannot size up.
- Every live ticket is logged (strike distance, time left, fair %, Kalshi price, result). A red close-No bucket turns that rule off.

Your Pi layout

- Code: `/home/KalshiBot`
- Venv: `/home/KalshiBot/.venv`
- Keys: `~/.kalshi/kalshi_private_key.pem` and the key id (file or `~/.kalshi/env`)
- Project env: `/home/KalshiBot/.env`

A two-line `.env` is enough:

```
USE_DEMO=false
SPOT_SOURCE=cfbenchmarks
```

Keys do not belong in `.env`. Leave `LIVE_TRADING` off. The timer confirms live on the command line.

`~/.kalshi/env` may use bash `export KEY=value`. systemd cannot read that. The bot loads it in Python.

Every session

```
cd /home/KalshiBot
source .venv/bin/activate
```

`./kb` uses the venv python when that venv exists.

Bot commands

Command	Also	What it does
`./kb`		Menu
`./kb scan`	s / 1	Report only. No orders.

`./kb scan --asset BTC`		BTC only
`./kb scan --asset ETH`		ETH only
`./kb once`	o / 2	Report + dry-run JSON. No orders.
`./kb auth`	a / 3	Test key + PEM
`./kb auth --prod`		Auth on live Kalshi
`./kb auth --demo`		Auth on demo
`./kb live`	l / 4	Real limits after you type LIVE
`./kb live --prod`		Live on prod. Type LIVE only if it says on PROD
`./kb live --demo`		Live on demo
`./kb live --prod --confirm LIVE`		Live, no prompt
`./kb env`	e / 5	Show DEMO vs PROD
`./kb env --prod`		Write USE_DEMO=false
`./kb env --demo`		Write USE_DEMO=true
`./kb --help`		Help

`--prod` / `--demo` also work on `scan` and `once`.

Daily use: `./kb scan` to look. `./kb live --prod` when you are at the keyboard and willing to rest a limit. Type LIVE only if the prompt says on PROD.

`NO_ACTIONABLE_EDGE` means sit. That is a successful run, not a crash.

Hourly timer (unattended live)

At minute 3 Eastern of every hour the Pi runs:

```
/home/KalshiBot/.venv/bin/python -m src.main live --prod --confirm LIVE
```

Same caps: one open idea, about $1.75, $2 max, post-only, far strikes only. If the Pi was off at :03, it does not fire late. If nothing clears 6% edge, it sits.

On (real money every hour):

```
sudo systemctl enable --now kalshi-hourly.timer
```

Off:

```
sudo systemctl disable --now kalshi-hourly.timer
```

Check:

```
systemctl is-enabled kalshi-hourly.timer
systemctl is-active kalshi-hourly.timer
systemctl list-timers kalshi-hourly.timer --no-pager
```

On = enabled and active. Off = disabled and inactive.

Fire one live run now (does not wait for :03):

```
sudo systemctl start kalshi-hourly.service
journalctl -u kalshi-hourly.service -n 80 --no-pager
```

See the unit:

```
systemctl show -p ExecStart kalshi-hourly.service --no-pager
```

After a code pull, reinstall:

```
sudo cp /home/KalshiBot/scripts/kalshi-hourly.service /home/KalshiBot/scripts/kalshi-hourly.timer /etc/systemd
sudo systemctl daemon-reload
```

Edit .env

```
nano /home/KalshiBot/.env
```

Save: Ctrl+O, Enter. Quit: Ctrl+X.

How to read the report

- Spot — CF Benchmarks BRTI/ERTI when the key works; Coinbase if not. Vol is exchange-realized.
- Actionable — the one idea: side, limit, fair, edge, size, max loss.
- Nearby watch — close, not enough edge.
- Avoid — failed a hard filter.

LIVE requote or LIVE post-only cross retry means the book moved; it stepped more passive. LIVE skipped means it would have taken; it did not lift. LIVE placed means a rest is on the book.

Git

```
git fetch origin
git pull
git checkout main
```

Health check

```
./kb auth --prod
```

Want: USE_DEMO=False, host external-api.kalshi.com, AUTH OK, a balance.

If something breaks

- 401 on demo — live key on the demo host. Use `--prod` or `./kb env --prod`.
- ModuleNotFoundError: pydantic — systemd used system python3. The unit must use `.venv/bin/python`. Recopy the service and daemon-reload.
- Ignoring invalid environment assignment — systemd cannot parse `export KEY=value`. The bot loads `~/.kalshi/env` itself. Recopy the latest service (no EnvironmentFile on that path).
- post only cross — book moved; newer code requotes one tick more passive.
- Binance 451 — US geo-block. CF Benchmarks first, then Coinbase. Vol still uses exchange candles.
- list-timers stuck in a pager — press q.

GitHub Actions is scan only. It does not place live orders. Cloud IPs often 403.

Rebuild this PDF

From the repo root, with DejaVu fonts installed:

```
python3 scripts/build_operating_manual.py
```

Writes `docs/KalshiBot-operating-manual.pdf`.